

Fundamentos de Comunicação



Arquitetura de Sistemas Distribuídos,
Paralelos e Concorrentes
Escola Politécnica – PUCPR
Luiz A. de P. Lima Jr. • luiz.lima@pucpr.br

1

Memória Compartilhada x Trocas de Mensagens

● IPC = Inter-Process Communication:

- ♦ **memória compartilhada:**
 - exige controle de acesso simultâneo a recursos compartilhados;
- ♦ **Sistemas Distribuídos:**
 - sincronização por meio do envio e recebimento de mensagens

● Mecanismos básicos de IPC usando mensagens:

- ♦ **Pipes** (variação: "**named pipes**" ou "FIFOs")
- ♦ **Sockets**

2



Entrada e Saída Padrão

Fundamentos de Comunicação

3

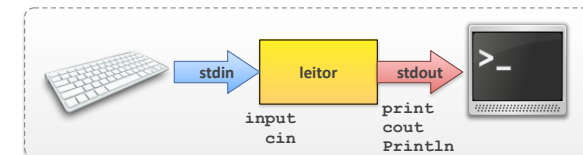
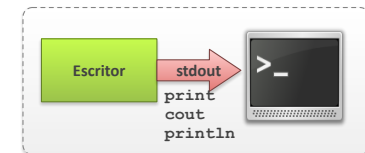
Entrada/Saída Padrão

● Redirecionamento de saída:

- ♦ **escritor > saída.txt**

● Redirecionamento de entrada:

- ♦ **leitor < entrada.txt**



4



Pipes e FIFOs



Fundamentos de Comunicação

5

Filtros

Exemplos de filtros:

- ✦ `cat`
- ✦ `tac`
- ✦ `tr`
- ✦ `head`
- ✦ `tail`
- ✦ `wc`
- ✦ `sort`
- ✦ `uniq`
- ✦ `sed`
- ✦ `grep`

- ✦ `cut`
- ✦ `rev`
- ✦ `tee`
- ✦ ...

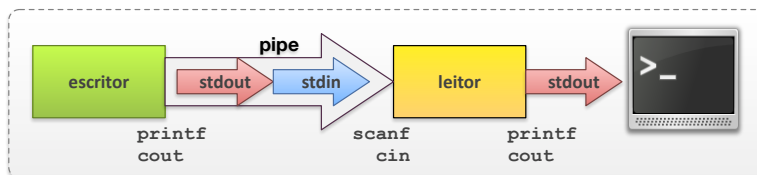
Utilize:
man cmd
para ver o manual do
comando "cmd"

6

Pipes

● Pipes:

- ✦ **unidirecionais**
- ✦ **prog1 | prog2**



7

7

Exercício Pipes

● Considere o arquivo: `/usr/include/stdio.h`

1. mostrar na tela todas as linhas do arquivo que contêm a string `"by"`;
2. mostrar na tela todas as linhas do arquivo que contêm a string `"else"`, não importando se as letras são minúsculas ou maiúsculas;
3. inverter os caracteres das linhas que contêm a string `"typedef"` mostrando-as na tela;
4. mostrar na tela o número de linhas que começam com a string `"typedef"`;
5. mostrar na tela o número de linhas que não terminam com dígitos;
6. substituir todas as ocorrências de `"typedef"` no arquivo pela string `"TIPO"` e mostrar na tela apenas as linhas onde a alteração foi feita;
7. mostrar na tela o número de linhas em que o quinto caractere é a letra `"e"`;
8. mostrar na tela quantas linhas repetidas o arquivo `/usr/include/stdio.h` possui.

8

8

Named Pipes (ou FIFOs)

● FIFOs:

- ✦ **estendem** o conceito tradicional de pipes
- ✦ **tipo especial de arquivo** que permite a comunicação entre 2 ou mais processos através da leitura/escrita neste "arquivo"
- ✦ `mkfifo`
 - ambas as pontas da comunicação (uma para a leitura e a outra para a escrita) precisam estar simultaneamente abertas para que seja possível enviar ou receber bytes da FIFO.

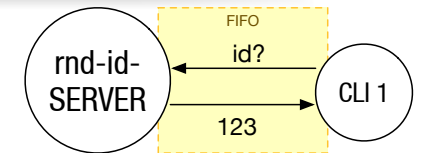
9

9

Exercício: FIFOs em Python

- ```
import os
ch = "canal"
os.mkfifo(ch)
e = os.open(ch, os.O_RDONLY)
msg = os.read(e, 128).decode("utf-8")
os.unlink(ch)
```
- ...
 

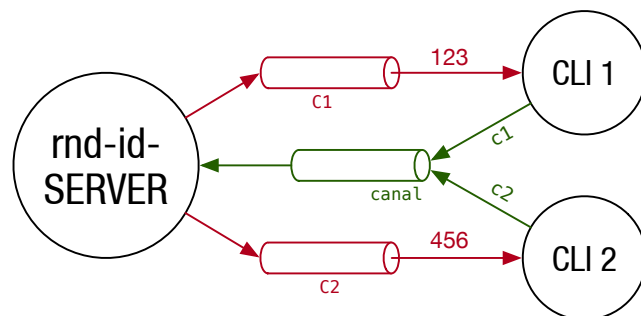
```
s = os.open(ch, os.O_WRONLY)
os.write(s, b"olá") # ou str.encode("ola")
```



10

10

## C/S com FIFOs (possível resposta)



11

11



ASDPC

## Teste Diagnóstico: Sockets

Fundamentos de Comunicação

12

## Diagnóstico Sockets

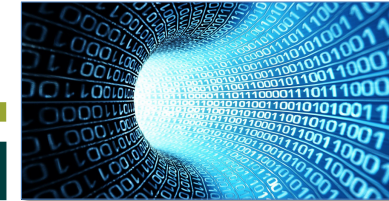
- [www.menti.com](http://www.menti.com)
- Quais são as **principais dificuldades** de se desenvolver aplicações complexas usando sockets diretamente?
  - ✦ Cuidado com **deadlocks**
  - ✦ Definição de **protocolo**
  - ✦ **Delimitação** de mensagens (TCP)
  - ✦ **Conversão** de formatos (*little-endian/big-endian*)
  - ✦ ...

13

13



ASDPC



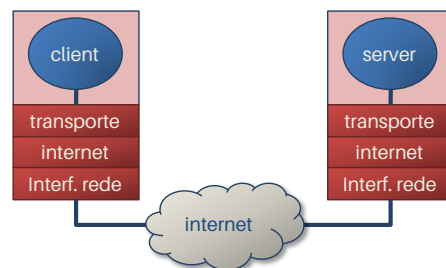
## Infraestrutura de Comunicação

Fundamentos de Comunicação

14

## O Paradigma Cliente/Servidor

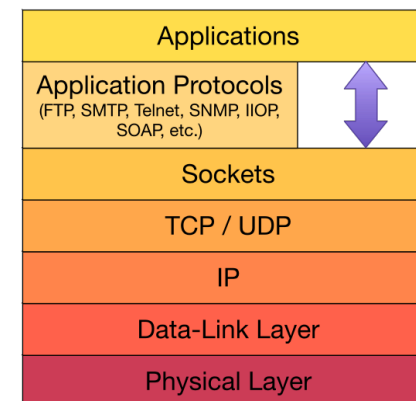
- Clientes e servidores usam o protocolo de transporte para comunicação.
- Transporte:
  - ✦ orientado a conexão (**TCP**)
  - ✦ não orientado a conexão (**UDP**)



15

15

## Sockets



Conjunto de APIs mais difundido para programação sobre a arquitetura TCP/IP.

16

16

## Sockets

- Desenvolvido para o UNIX versão Berkeley
  - ✦ Método padrão para **conectar duas máquinas numa rede**
  - ✦ Oferece mecanismos para manipular o **fluxo de dados bidirecional** em uma conexão de rede virtual
  - ✦ **Padrão** atual da Internet

17

17

## Sockets

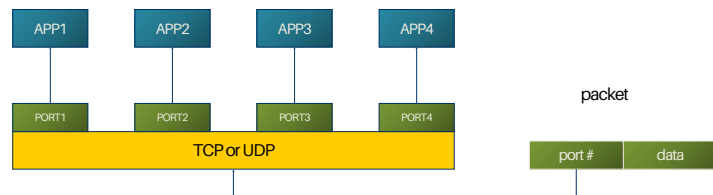
- Suportam serviços para:
  - ✦ protocolos orientados a conexão (**TCP**)
  - ✦ protocolos não-orientados a conexão (**UDP**)
- Usam o paradigma:
  - ✦ `open` / `read-write` / `close`para entrada e saída (típico do UNIX).

18

18

## Portas

- Tipicamente:
  - ✦ O número de processos com comunicação na rede em uma máquina **excede o número de conexões físicas** da máquina com a rede.
- **Portas:**
  - ✦ para enviar dados a **processos específicos** rodando no computador.



19

19

## Princípio

- Sockets podem ser utilizados para efetuar comunicação sobre os protocolos **TCP ou UDP**.

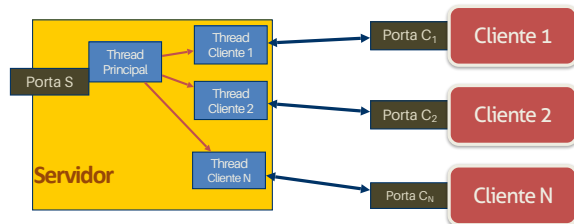


20

20

## Mais de um cliente

- O servidor pode utilizar a **mesma porta** para estabelecer várias conexões.
- Cada cliente deve ser **atendido por um thread** separadamente.



21

21

## Portas e Sockets

- **Par de sockets** em conexão TCP é formado por uma **quatro-tupla**:

{ IP Local:Porta local, IP Remoto:Porta Remota }

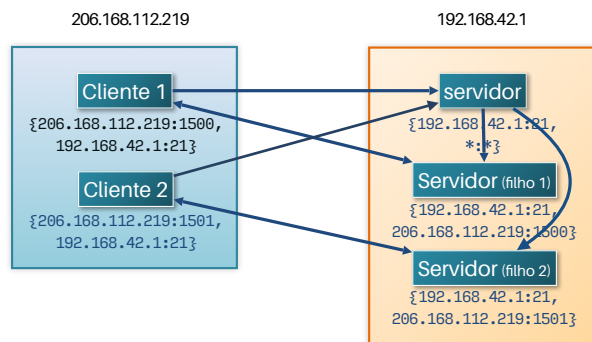
- **Par de sockets:**

- ✦ identifica a conexão de forma única na rede.

22

22

## Portas e Sockets



23

23

## Portas e Sockets

- **Multiplexação do TCP:**

- ✦ Não somente por número de porta destino.
- ✦ Cada **elemento do par de sockets** é examinado para determinar qual extremidade receberá os dados.

24

24

## Convenções de portas

- Portas de **servidor** têm números < 1024
- Portas de **cliente** tem número >= 1024
- Até **64K portas** são possíveis em um servidor
- Portas padronizadas por RFCs:
  - ✦ **22/TCP**: SSH
  - ✦ **25/TCP**: SMTP
  - ✦ **80/TCP**: HTTP
  - ✦ **161/UDP**: SNMP
  - ✦ ...

25

25



ASDPC

## Prática: NetCat



26

## NetCat (nc)

- Aplicativo para conexões TCP/UDP:
  - ✦ Abrir conexões TCP
  - ✦ Enviar datagramas (UDP)
  - ✦ Ouvir conexões UDP/TCP em portas arbitrárias
  - ✦ Etc.
  - ✦

27

27

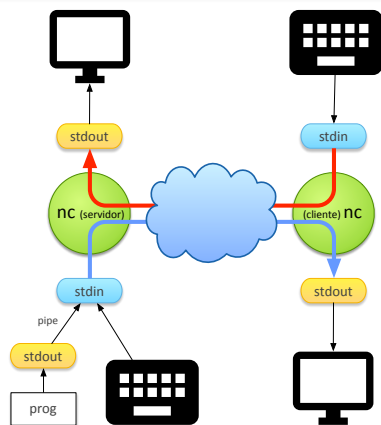
## NetCat (nc)

- Opções:
  - ✦ **-l**: ouve na porta
  - ✦ **-u**: usa UDP ao invés de TCP
  - ✦ **-v**: modo "verbose"
  - ✦ (lista completa: `man nc`)
- Entrada padrão: `stdin`
- Saída padrão: `stdout`

28

28

## NetCat (nc)



29

29

## NetCat: Exercício

- Crie um esquema de “redirecionamento” de comunicação de origem (O) a destino (D) usando NetCat e pipes conforme figura abaixo:



30

30

## Leitura Adicional

- W. R. R. Stevens et al. “**Programação de Rede UNIX**”, Bookman, 3a edição.
- Daniel J. Barrett, Robert G. Byrnes, Richard E. Silverman, “**SSH, the Secure Shell**”, O’Reilly
- Orfali, “**Client/Server Survival Guide**”, Willey.
- Comer, “**Computer Networks and Internets**”, Prentice Hall.
- G. Coulouris et al. “**Distributed System: Concepts and Design**”, 5th edition, Addison Wesley, 2012.
- Tanenbaum, A. “**Sistemas Distribuídos**”, Wiley, 2010.

31

31